

CS 470/547: Assignment 01

Programming Assignments (80%)

Your program **MUST** be written in **Python**.

Your code file should be named **Assign01.py**. The goal of this assignment is to perform various intensity transformations.

Restrictions

You CANNOT use the following functions:

- **cv2.equalizeHist**
- **cv2.calcHist**
- **cv2.LUT**
- **Anything from the cv2.intensity_transform module**
- **np.histogram**

You **MAY** use np.bincount, np.cumsum, np.sum, np.interp, np.arange, np.clip, np.log, np.round and other np functionality. ***If you are not sure whether a library function is permitted, ASK ME!***

For efficiency reasons, while and for loops must be avoided. Everything can be implemented by taking advantage of numpy's indexing and broadcasting functionality.

Assign01.py

For CS 470 and 547

All intensity transformation creation functions should return a numpy array of length 256 with dtype "uint8". **This is a lookup table (LUT).** ***You must also ensure that floating values are rounded, clipped to [0, 255], and converted to unsigned bytes.***

All input images are grayscale images of shape (height, width) and with dtype "uint8". Output images are expected to be the same.

You are welcome and in fact encouraged to write helper functions for this assignment, BUT the below functions should be written as specified:

- **def get_log_transform(max_r)**
 - **max_r** = The maximum input value that will map to 255
 - Compute the corresponding c value: $c = \frac{255}{\log(1+max_r)}$
 - Return the appropriate LUT for the log transform:
$$s = c \log(1 + r)$$

- **def get_gamma_transform(gamma)**
 - **gamma** = The gamma exponent
 - Return the appropriate LUT for the gamma/power-law transform:
$$s = 255 * \left(\frac{r}{255}\right)^{\text{gamma}}$$
- **def get_hist_equalize_transform(image, do_stretching)**
 - **image** = Grayscale unsigned byte image
 - **do_stretching** = If True, do histogram stretching
 - Return the appropriate LUT for histogram equalization.
- **def get_piecewise_linear_transform(points)**
 - **points** = A *2D Python list* of (r, s) points that make up a piecewise linear transformation.
 - *HINT*: Sort the points by r , take advantage of zip to get lists of r and s values, and leverage np.interp to get the intermediate points.
 - Return the appropriate LUT for the specified piecewise linear transformation.
- **def apply_intensity_transform(image, int_transform)**
 - **image** = Grayscale unsigned byte image
 - **int_transform** = unsigned byte LUT (as a numpy array)
 - Return the transformed image (should be the same shape and type as original image).

For CS 547 Only

In addition to the preceding requirements, you will write functionality to guess the gamma exponent from an input and output image.

- **def estimate_gamma_exponent(image, output)**
 - **image** = Grayscale unsigned byte image
 - **output** = Grayscale unsigned byte image from applying a gamma transformation on input image
 - Estimate and return the gamma exponent used to generate the output image.
 - To accomplish this, you are largely free to use any approach you see fit, and I encourage you to explore how this may be estimated.
 - One thing to consider is that you will need to get the current LUT from image to output. Since not all values may have a mapping, I suggest you also keep track of a mask array that tells you which values are valid.

Testing Screenshot (10%)

For CS 470 and 547

I have provided several files for testing:

- **Test_Assign01.py** – the test program
- **General_Testing.py** – basic testing functionality
- **assign01/**
 - o **images/**
 - Sample input images
 - o **output/**
 - Corresponding output images from various transformations

Once you have merged from my repo into yours, these files/folders should reside in your main project directory. Your source file (Assign01.py) should reside in the main project directory as well (NOT the assign01 folder).

You can either run the testing programs directly OR you can use the testing section of Visual Code.

You MUST run Test_Assign01.py and send a screenshot of the test results! Even if your program(s) do not pass all the tests, you MUST send this screenshot!

You may have to do "Command Palette" → "Python: Configure Tests" → unittest → root directory → Test_*.py

This screenshot should show clearly:

- The final result of the test run on the command line ("OK" for all passing, "FAILED (failures=N)" for some or all failing).
- OR
- The testing view in Visual Code (see image on right)

The screenshot should be copied to the screenshots/ folder.

WARNING: Passing the tests does NOT guarantee your assignment code is working 100% correctly! You are responsible for manually verifying whether your output visually matches what it ought to be.

For CS 547 Only

I have also provided the following:

- **Eval_Assign01_Grad.py** – calculates and prints the EMD for the input/output image histograms
- **assign01/**
 - o **gamma/**
 - Corresponding output images after mystery gamma transformation(s)

You do NOT need to send a screenshot of the result of Eval_Assign01_Grad.py.

For the Eval_Assign01_Grad script, you should be shooting for an AVERAGE loss of under 0.3. Bonus credit will be available for the LOWEST loss in the class (INCLUDING my own SECRET evaluation numbers 😊)

README.md (10%)

Update the README.md of your repository with the following:

- Change "YOUR NAME HERE" to your actual name.
- Add a new entry under "Runnable Python Scripts" (similar to the one already there for BasicVision.py) for "Assign01.py". Very briefly describe what functionality is available. I am only looking for a few sentences.
 - o **CS 547 Only:** In addition, **briefly describe the approach** you implemented for the gamma estimation task, and **what results you got**.

Grading

Your OVERALL assignment grade is weighted as follows:

- 10% - Testing results screenshot
- 10% - README update
- 80% - Programming assignments

I reserve the right to take points off for not meeting the specifications in this assignment description.

CS 470 Only

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	Maximum Penalty
Code that is not syntactically correct	60
Usage of forbidden function(s)	60
One GIGANTIC commit for whole assignment	30
Incorrectly implemented LUT generation functions	60
Usage of for/while loops	10
Incorrectly implemented apply_intensity_transform() function	10
Missing testing screenshot	10
README update missing/incorrect	10
Technically correct code, but poor code design	5
Missing assignment Git branch	5
Failed to merge assignment branch	5
Wrong names for branch, files, etc.	2 for each
Bad commit messages	2 for each

CS 547 Only

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	Maximum Penalty
Code that is not syntactically correct	60
Usage of forbidden function(s)	60
One GIGANTIC commit for whole assignment	30
Incorrectly implemented LUT generation functions	45
Incorrect and/or poor loss on gamma estimation	15
Usage of for/while loops	10
Incorrectly implemented <code>apply_intensity_transform()</code> function	10
Missing testing screenshot	10
README update missing/incorrect	10
Technically correct code, but poor code design	5
Missing assignment Git branch	5
Failed to merge assignment branch	5
Wrong names for branch, files, etc.	2 for each
Bad commit messages	2 for each

Bonus credit is available for this assignment: note that bonus credit will only be awarded if functionality is correctly implemented:

- (5%) Lowest loss on public AND secret gamma estimation